



Unidad 05 – Bucles

Cátedra: Programación en Computación – UTN FRRQ **Docentes:** Longhi Pablo, Talijancic Iván



De qué se trata esta unidad

Todos tus programas hasta ahora procesan **una** medición. Si el tablero tiene treinta, tenés treinta programas o un programa de mil líneas.

Hoy aprendés a decirle a la computadora "**hacé esto treinta veces**".

Al terminar vas a poder:

- Repetir un bloque con `for`, `while` y `do-while`
- Elegir **cuál de los tres** conviene en cada caso
- Usar el patrón **acumulador** para sumar y promediar
- Usar el patrón **contador** para contar los casos que cumplen una condición
- Encontrar el **máximo** y el **mínimo** de una serie
- Reconocer —y salir de— un **bucle infinito**



Los patrones de hoy son la base de todo lo que viene. En la unidad 07, cuando aparezcan los vectores, vas a recorrerlos con exactamente estos bucles y estos patrones. Si hoy queda flojo, ahí se nota.

1. El problema

Tres mediciones, tres líneas. Se banca:

```
console.log('Medición 1')
console.log('Medición 2')
console.log('Medición 3')
```

¿Y las treinta y un mediciones de un mes?

```
console.log('Medición 1')
console.log('Medición 2')
// ... 28 líneas más ...
console.log('Medición 31')
```

Inviabile. Y el problema de fondo todavía no apareció: **¿y si el usuario decide cuántas son?** No podés escribir a mano una cantidad de líneas que se conoce recién cuando el programa corre.

Un **bucle** resuelve las dos cosas.

2. **for** : cuando sabés cuántas veces

```
for (let i = 1; i <= 3; i++) {  
  console.log(`Medición ${i}`)  
}
```

```
Medición 1  
Medición 2  
Medición 3
```

Tres líneas de código, tres repeticiones. Cambiá el `3` por `31` y son treinta y una, sin escribir una línea más.

2.1 Las tres partes

Dentro del paréntesis hay **tres cosas separadas por punto y coma**:

```
for (let i = 1; i <= 3; i++) {  
//      ↑      ↑      ↑  
//      |      |      | — incremento: qué pasa al final de cada vuelta  
//      |      |      | — condición: mientras sea true, sigue  
//      |      |      | — inicialización: se ejecuta UNA vez, al principio  
}
```

✖ Ese `i` es la **variable de control**. Se llama así por *índice*, y por costumbre se le dice `i`. Es la única variable del curso a la que se le permite un nombre de una letra: todo el mundo la reconoce.

2.2 El orden exacto en que pasan las cosas

Es lo que más confusión genera. La secuencia es:

1. **Inicialización** (una sola vez)
2. **¿Condición?** Si es `false`, el bucle **termina**
3. **Cuerpo** (las líneas entre llaves)
4. **Incremento**
5. Volver al paso 2

2.3 La traza: seguirlo paso a paso

Cuando un bucle no hace lo que esperás, **escribí la traza**: una tabla con el valor de cada variable en cada vuelta. Es la herramienta más efectiva que hay para depurar bucles.

```
for (let i = 1; i <= 4; i++) {
  console.log(i)
}
```

VUELTA	I AL ENTRAR	¿ I <= 4 ?	IMPRIME	I DESPUÉS DE I++
1	1	✓ sí	1	2
2	2	✓ sí	2	3
3	3	✓ sí	3	4
4	4	✓ sí	4	5
—	5	✗ no	—	el bucle termina

Fijate en la última fila: **el cuerpo no se ejecuta**. La condición se evalúa *antes*, y al dar `false` el bucle corta sin entrar.

💡 **Hacé la traza en papel.** En serio. Dos minutos de tabla te ahorran veinte de mirar la pantalla.

2.4 De cuánto en cuánto

El incremento no tiene por qué ser de a uno:

```
for (let i = 0; i <= 20; i += 5) {
  console.log(i)
}
```

```
0
5
10
15
20
```

Ni tiene que ir para arriba:

```
for (let i = 5; i >= 1; i--) {
  console.log(i)
}
```

```
5
4
3
2
1
```

3. `while` : cuando no sabés cuántas veces

`while` repite **mientras** una condición sea verdadera. No lleva inicialización ni incremento adentro: los ponés vos.

```
let pressure = 10

while (pressure > 6) {
  console.log(`Presión: ${pressure} bar`)
  pressure = pressure - 1
}
```

```
Presión: 10 bar
Presión: 9 bar
Presión: 8 bar
Presión: 7 bar
```

3.1 Las tres piezas siguen estando

Lo que en el `for` iba junto, en el `while` queda repartido:

```
let i = 1           // 1. inicialización – ANTES del bucle

while (i <= 3) {     // 2. condición
  console.log(i)
  i++               // 3. incremento – ADENTRO, y no te lo podés olvidar
}
```

⚠ Ese `i++` es obligatorio. Si lo olvidás, `i` vale `1` para siempre, la condición nunca se hace falsa y el programa **no termina nunca**. Es el error más común de la unidad.

4. `do-while` : al menos una vez

Igual que `while`, pero la condición se evalúa **al final**. Consecuencia: el cuerpo se ejecuta **siempre al menos una vez**.

```
let value = 100

do {
  console.log(`Valor: ${value}`)
  value = value - 30
} while (value > 0)
```

```
Valor: 100
Valor: 70
Valor: 40
Valor: 10
```

4.1 Para qué sirve de verdad

Para **pedir un dato y validarlo**: primero tenés que pedirlo, y recién después podés saber si sirve.

```
import { prompt } from './prompt.js'

let temperature

do {
  temperature = parseFloat(prompt('Temperatura [°C] (0 para terminar): '))
  console.log(`Registrada: ${temperature} °C`)
} while (temperature !== 0)
```

Ese `0` que corta el bucle se llama **centinela**: un valor acordado que significa "no hay más datos". Es el patrón clásico para cargar una cantidad de datos que no se sabe de antemano.

⚠ Fijate que `temperature` se declara **antes** del `do`, con `let` y sin valor. Si la declarás adentro, al llegar al `while` la variable ya no existe: nació dentro del bloque y muere con él (unidad 03).

5. 🧭Cuál usar

SITUACIÓN	BUCLE
Sé exactamente cuántas veces (10 mediciones, del 1 al 31)	<code>for</code>
Depende de una condición que cambia (mientras la presión baje)	<code>while</code>
Hay que ejecutarlo al menos una vez (pedir un dato y validarlo)	<code>do-while</code>

En la duda, `for`. Es el que menos se olvida de incrementar, porque el incremento está a la vista en la primera línea.

🔑 Los tres son **intercambiables**: cualquier cosa que hagas con uno la podés hacer con los otros dos. La elección es de **legibilidad**, no de capacidad.

6. + El patrón acumulador

El patrón más importante del curso. Sirve para sumar, para promediar, y en la unidad 07 lo vas a usar en cada ejercicio de vectores.

Son tres pasos, siempre los mismos:

```
let total = 0 // 1. ANTES del bucle: arranca en cero

for (let i = 1; i <= 5; i++) {
  total = total + i // 2. ADENTRO: se le suma algo en cada vuelta
}

console.log(total) // 3. DESPUÉS: el resultado
```

15

La traza deja ver por qué funciona:

VUELTA	I	TOTAL ANTES	TOTAL = TOTAL + I
1	1	0	1
2	2	1	3
3	3	3	6
4	4	6	10
5	5	10	15

6.1 ⚠ El acumulador va afuera

Es el error que más cuesta ver:

```
// ❌ Así no
for (let i = 1; i <= 5; i++) {
  let total = 0
  total = total + i
}
```

Declarado adentro, **total** **nace y muere en cada vuelta**. Empieza en **0** cinco veces y el resultado se pierde. Además, al salir del bucle la variable ya no existe.

6.2 Promediar

Sumar y dividir por la cantidad:

```
const MEASUREMENTS = 4
let total = 0

for (let i = 1; i <= MEASUREMENTS; i++) {
  total = total + i * 100
}

console.log(`Promedio: ${total / MEASUREMENTS}.toFixed(2)}`)
```

Promedio: 250.00

⚠ La división va **después** del bucle, no adentro. Adentro estarías promediando cada vuelta con datos incompletos.

7. El patrón contador

Igual que el acumulador, pero en vez de sumar el valor **suma uno**, y sólo cuando se cumple una condición. Acá se juntan los bucles de hoy con los condicionales de la unidad 04:

```
const LOWER_LIMIT = 361
const UPPER_LIMIT = 399
let outOfRange = 0

for (let i = 1; i <= 5; i++) {
  const measured = 350 + i * 15

  if (measured < LOWER_LIMIT || measured > UPPER_LIMIT) {
    outOfRange = outOfRange + 1
  }
}

console.log(`Fuera de rango: ${outOfRange}`)
```

Fuera de rango: 2

Las cinco mediciones son 365, 380, 395, 410 y 425. Las dos últimas se pasan del límite superior.

💡 `outOfRange++` hace exactamente lo mismo que `outOfRange = outOfRange + 1`, y es lo que vas a ver escrito en todos lados.

8. Máximo y mínimo

Otro patrón que vuelve en cada ejercicio de vectores. La idea: guardás un "campeón" y lo vas reemplazando cada vez que aparece algo mejor.

```
let maximum = 0

for (let i = 1; i <= 5; i++) {
  const measured = 350 + i * 15

  if (measured > maximum) {
    maximum = measured
  }
}

console.log(`Máxima: ${maximum}`)
```

```
Máxima: 425
```

8.1 La trampa de inicializar en cero

Ese `let maximum = 0` funciona **de casualidad**, porque todas las mediciones son positivas. Probá con temperaturas bajo cero:

```
let maximum = 0 // ❌
// mediciones: -5, -12, -3 → el máximo real es -3
```

El programa contesta `0`, que **no es ninguna de las mediciones**. Nunca hubo un `-3 > 0`.

La forma correcta: inicializar con el **primer valor real**, no con un número inventado.

```
let maximum = firstMeasurement // ✅ el primero de la serie
```

 En la unidad 07 esto se escribe `maximo = vector[0]`, y es **exactamente el mismo razonamiento**. Se corrige como error grave, así que conviene entenderlo ahora.

9. Leer varios datos por consola

Juntando todo: pedir cuántas mediciones son, leerlas una por una y promediar.

```
import { prompt } from './prompt.js'

const howMany = parseInt(prompt('¿Cuántas mediciones? '))
let total = 0

for (let i = 1; i <= howMany; i++) {
  const measured = parseFloat(prompt(`Medición ${i}: `))
  total = total + measured
}

console.log(`Promedio: ${total / howMany.toFixed(2)}`)
```

```
¿Cuántas mediciones? 3
Medición 1: 380
Medición 2: 372.5
Medición 3: 391
Promedio: 381.17
```

Ese es el esqueleto de casi todo el TP. Leer, acumular, mostrar.

10. 🤖 Bucles anidados

Un bucle adentro de otro. El de adentro **completa todas sus vueltas por cada vuelta** del de afuera.

```
for (let fila = 1; fila <= 2; fila++) {
  for (let columna = 1; columna <= 3; columna++) {
    console.log(`fila ${fila}, columna ${columna}`)
  }
}
```

```
fila 1, columna 1
fila 1, columna 2
fila 1, columna 3
fila 2, columna 1
fila 2, columna 2
fila 2, columna 3
```

Dos vueltas por tres vueltas: **seis** líneas. Las variables de control se llaman distinto (`fila` y `columna` , o `i` y `j`) — si las dos se llamaran `i` sería un desastre.

🔪 Por ahora alcanza con reconocerlos. En la **unidad 08**, con matrices, van a ser el pan de cada día.

11. ∞ Bucles infinitos

Un bucle cuya condición **nunca se hace falsa**. El programa se cuelga y no termina más.

```
let i = 1

while (i <= 5) {
  console.log(i)
  // ❌ falta el i++
}
```

Eso imprime `1` para siempre.

Cómo salir: `Ctrl + C` en la terminal. Es la misma combinación con la que cortabas `npm run dev` en la unidad 02.

Las tres causas, en orden de frecuencia:

1. **Falta el incremento** — `while` sin `i++`
2. **El incremento va para el lado equivocado** — `i--` con la condición `i <= 10`
3. **La condición nunca puede ser falsa** — `while (i > 0)` con `i` que sólo crece

⚠️ Si tu programa imprime miles de líneas o queda colgado sin devolver el cursor: `Ctrl + C` y revisá el **incremento**. No está roto Node, está roto el bucle.

12. 🐛 Errores comunes

ERROR	SÍNTOMA	SOLUCIÓN
Falta el incremento en <code>while</code>	El programa no termina	<code>Ctrl + C</code> y agregó el <code>i++</code>
Acumulador declarado adentro	El total siempre da lo del último paso	Declaralo antes del bucle
<code><</code> donde va <code><=</code>	Falta la última vuelta	Contá con la traza
Empezar en <code>0</code> cuando se cuenta desde <code>1</code>	Sobra una vuelta o falta	Elegí uno y sé consistente
Máximo inicializado en <code>0</code>	Falla con valores negativos	Inicializá con el primer valor real
Dividir adentro del bucle	El promedio da cualquier cosa	La división va después
Declarar la variable del <code>do-while</code> adentro	<code>ReferenceError</code> en la condición	Declarala antes, con <code>let</code>
Dos bucles anidados con la misma <code>i</code>	Resultados imposibles	<code>i</code> y <code>j</code> , o nombres descriptivos

13. 📋 Resumen

ESTRUCTURA	CUÁNDO
<code>for (let i = 0; i < n; i++)</code>	Sé cuántas veces
<code>while (condición)</code>	Mientras se cumpla algo
<code>do { } while (condición)</code>	Al menos una vez — validar, centinela

Los tres patrones que hay que saber de memoria:

```
let total = 0 // ACUMULADOR
for (...) { total = total + valor }

let cuantos = 0 // CONTADOR
for (...) { if (condición) { cuantos++ } }

let maximo = primerValor // MÁXIMO
for (...) { if (valor > maximo) { maximo = valor } }
```

🔑 Los tres tienen la misma forma: **declarar antes, actualizar adentro, usar después.**

14. 🛠️ Para la próxima clase

En la **unidad 06** llegan las **funciones**: darle un nombre a un pedazo de código para usarlo muchas veces sin repetirlo. Es lo que va a hacer que los programas del TP de hoy —que son largos y repetitivos— entren en la mitad de líneas.

También aparece `Math.random()`, para generar mediciones sin tener que tipearlas una por una.

Para llegar preparado:

1. ✅ Hacé el TP, sobre todo los problemas **6** y **7**
 2. ✅ Asegurate de poder escribir los **tres patrones de memoria**, sin mirar
 3. ✅ Practicá la **traza** en papel: es lo que más se usa para depurar en el parcial
-

📎 Material de la unidad

- [Presentación de clase](https://github.com/italijancic/pc-2026/blob/main/unidades/05-bucles/presentacion.pdf) (https://github.com/italijancic/pc-2026/blob/main/unidades/05-bucles/presentacion.pdf)
- [Trabajo Práctico](https://github.com/italijancic/pc-2026/blob/main/unidades/05-bucles/tp.pdf) (https://github.com/italijancic/pc-2026/blob/main/unidades/05-bucles/tp.pdf)
- [Ejemplos de la clase](https://github.com/italijancic/pc-2026/tree/main/unidades/05-bucles/ejemplos) (https://github.com/italijancic/pc-2026/tree/main/unidades/05-bucles/ejemplos)